

Harbor + Terraform + CI sécurisée

Objectif : déployer un registre d'images privé (Harbor) par Infrastructure as Code, sécuriser sa chaîne de livraison, et la rendre observable.

1 IaC

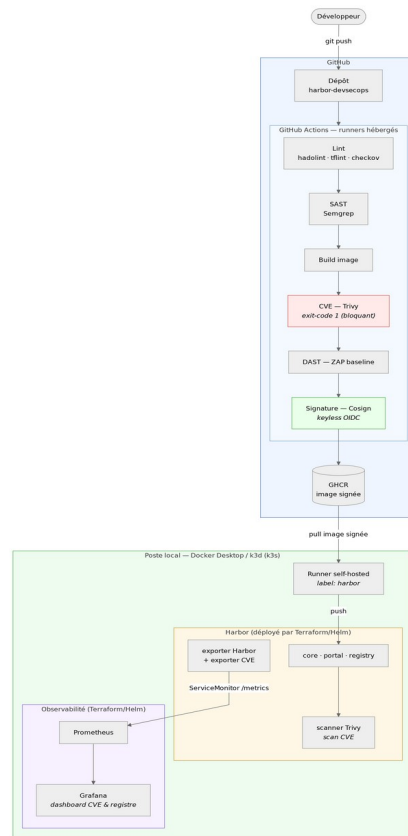
Terraform + Helm sur k3s

2 CI sécurisée

lint·SAST·CVE·DAST·signature

3 Observabilité

Prometheus + Grafana



Les concepts DevSecOps

Grille : *Expliquer les concepts · Comprendre où ils s'intègrent en production*

- ▶ DevSecOps = intégrer la sécurité DANS la chaîne CI/CD, automatisée et au plus tôt (« shift-left »).
- ▶ Avant le build : lint + SAST (code). Sur l'artefact : scan CVE + DAST. Avant publication : signature.
- ▶ Dans le registre : re-scan continu. En production : observabilité des vulnérabilités.
- ▶ Une porte qui échoue BLOQUE la livraison — la sécurité est présente à chaque phase, pas à la fin.

À dire : « DevOps = livrer vite ; DevSecOps = livrer vite ET sûr, avec des contrôles automatiques à chaque étape. »

COMPÉTENCE 3

Infrastructure sécurisée par IaC

Grille : *Maîtriser une infrastructure sécurisée (K8s) via de l'IaC*

- ▶ Cluster k3s (via k3d) décrit en IaC : Terraform pilote Helm pour déployer Harbor + la supervision.
- ▶ Isolation par namespaces (harbor / monitoring) ; infra reproductible, versionnée, idempotente.
- ▶ Conteneurs durcis : utilisateur non-root, FS en lecture seule, capacités supprimées, sondes.
- ▶ Secrets jamais en clair (set_sensitive / secret Kubernetes) ; registre Harbor privé.

À dire : « Mon infra est en IaC, isolée, durcie, avec des secrets protégés et un registre privé. »

Pipeline CI/CD : sécuriser l'artéfact

Grille : Sécuriser une pipeline CI/CD permettant de sécuriser la création d'artéfact

1

Lint

Dockerfile + IaC

2

SAST

Semgrep (code)

3

CVE

Trivy — exit-code 1

4

DAST

ZAP (appli)

5

Signature

Cosign keyless

→ l'artéfact (l'image) n'est publié que s'il passe toutes les portes, puis il est signé.

À dire : « Chaque image passe 5 portes ; un échec (ex. Trivy exit-code 1) bloque. L'artefact livré est contrôlé et signé. »

COMPÉTENCE 4

Observabilité orientée sécurité

Grille : Adapter l'observabilité à un contexte DevSecOps

- ▶ Chaîne : Harbor expose → ServiceMonitor (CRD) → Prometheus collecte/stocke → Grafana affiche.
- ▶ Observabilité tournée SÉCURITÉ : le dashboard suit les vulnérabilités (CVE) par sévérité, pas que la santé.
- ▶ Un exporter maison interroge l'API Harbor pour exposer le détail des CVE en n...



À dire : « J'ai orienté la supervision vers la sécurité : mon dashboard suit les CVE par sévérité. »

COMPÉTENCE 7

Obtenir & communiquer la sécurité

Grille : *Obtenir et communiquer les informations de sécurité au sein d'une organisation*

Obtenir

- Scans de vulnérabilités : Trivy en CI + scanner intégré de Harbor
- Métriques CVE collectées par Prometheus

Communiquer

- Dashboard Grafana partagé (visible par l'équipe)
- Rapports téléchargeables (ZAP, UI Harbor)
- Signature vérifiable par un tiers (cosign verify)
- Évolution possible : alertes Alertmanager

À dire : « J'obtiens l'info via les scans, je la communique via le dashboard partagé, les rapports et la signature vérifiable. »

Couverture de la grille d'évaluation

Compétence évaluée	Preuve dans mon projet
Expliquer les concepts DevSecOps	Shift-left + pipeline bloquante
Où s'intègrent les pratiques en prod	Sécurité à chaque étape du cycle CI/CD
Maîtriser une infra sécurisée (IaC)	k3s par Terraform, durci, secrets protégés
Adapter l'observabilité au DevSecOps	Dashboard CVE (Prometheus/Grafana)
Sécuriser la pipeline / l'artéfact	5 portes + signature Cosign
Déployer des outils de sécurisation	hadolint, checkov, Semgrep, Trivy, ZAP, Cosign, Harbor
Obtenir & communiquer la sécurité	Scans → dashboard partagé, rapports, signature